

Programmeerimise 2. kontrolltöö

- Punktiline osa: kaks ülesannet.
 - Materjalide kasutamine on **lubatud**.
 - **Lahendused** ning **logid** laadida Moodle'isse, mõlemale on eraldi lingid.
 - Ilma logi esitamata lahendused arvesse ei lähe.
- Kokku on kontrolltööks aega **90 minutit**.
- Igasugune suhtlemine on **keelatud**.

Enne punktilise osaga alustamist

Kasuta programmeerimiseks Thonnyt! Palun ära kirjuta programmi teises IDEs ega kopeeri seda sealt Thonnyssse! Sel juhul paistab logisid vaadates, et programm on kusagilt *copy-paste*'iga saadud, mis võib olla plagiaat. Veendu enne ülesandega alustamist, et logimine on sisse lülitatud (*Tools* → *Options* → *General* → *Log program usage events* või *Tööriistad* → *Seaded* → *Üldine* → *Logi kasutussündmusi*). Selleks, et logimine sujuks õigesti, **sulge enne ülesannetega alustamist Thonny ning käivita uuesti**. See tagab, et logisse ei jää üleliigset infot, kui Thonny on kauem avatud olnud.

Töö esitamine

Kui ülesanded on valmis, siis sulge korra Thonny ning käivita uuesti, et uusimad logid kindlasti salvestuksid. Esita programmid ja Thonny logifail(id). Programmidel võivad olla vabalt valitud nimed.

- Programmid esita Moodle'is lingi alt *2. kontrolltöö programmeerimise esitamine*.
- Logid leiad Thonny logifailide kaustast: ava menüü *Tools* → *Open Thonny data folder* (*Tööriistad* → *Ava Thonny andmekast*) ja seejärel ava kaust *user_logs*. Vali kõik failid, mis on tekkinud kontrolltöö ajal (kui vahepeal Thonnyt rohkem ei suletud, siis on neid faile vaid üks) ning esita need Moodle'is lingi alt *2. kontrolltöö logide esitamine*.

Ülesanne 1. Püropood (10 p)

103 aastat tagasi kuulutas Läti Vabariik end iseseisvaks. Sünnipäeva puhul ostetakse kauplustest palju pürotehnikat. Et arvepidamine segi ei läheks, koostas kauplus kogu pürotehnika kohta hinnakirja, mis asub failis *hinnakiri.txt*. Failis on igal real komadega eraldatult toote nimi sõnena, hind ujukomaarvuna ja toote värvus sõnena. Ridade arv failis pole ette teada.

Näide faili *hinnakiri.txt* võimalikust sisust

```
taevalatern,3.80,kuldne
raketid,14.10,roheline
taevalatern,2.70,punane
```

Kirjuta funktsioon `loe_failist`, mis võtab argumendiks faili nime ning tagastab sõnastiku, kus võtmeks on toote nimi ning väärtuseks on sõnastik, kus on kirjas toote värvused ja hinnad. Selle sõnastiku võtmeks on värvus ning väärtuseks toote hind ujukomaarvuna. Samanimeliste toodete värvused ja hinnad tuleb lisada sisemisse sõnastikku.

Näide funktsiooni `loe_failist` tööst

```
>>> loe_failist("hinnakiri.txt")
{'taevalatern': {'kuldne': 3.8, 'punane': 2.7}, 'raketid':
{'roheline': 14.1}}
```

Põhiprogrammis tuleb funktsiooni `loe_failist` abil sisse lugeda andmed failist *hinnakiri.txt*. Seejärel peab programm väljastama eraldi ridadele hinnakirjas olevad tooted koos hinna ja värvusega, mis on sulgudes (soovitav teha eraldi funktsioon). Järgmiseks küsib programm kasutajalt, millist toodet ta soovib osta. Kui selle nimega toode on olemas, tuleb küsida kasutajalt värvust. Kui toode või vastava värvusega toode puudub, siis antakse sellest kasutajale teada ning küsitakse uuesti toote nime. Pärast igat edukat toote ostukorvi lisamist väljastatakse kasutajale valitud kaupade jooksev summa. Tooteid küsitakse seni kuni kasutaja sisestab tühja sõne (""). Seejärel väljastab programm tasumisele kuuluva kogusumma ning lõpetab töö.

Näide põhiprogrammi tööst (kasutaja sisend on paksus kaldkirjas)

```
Kaupluses on müügil järgmised tooted:
taevalatern - 3.8€ (kuldne)
taevalatern - 2.7€ (punane)
raketid - 14.1€ (roheline)
Sisesta ostusoov: taevalatern
Sisesta toote värvus: sinine
Sellise värvusega toodet ei ole.
Sisesta ostusoov: taevalatern
Sisesta toote värvus: punane
Jooksev summa on 2.7€.
Sisesta ostusoov: raketid
Sisesta toote värvus: roheline
Jooksev summa on 16.8€.
Sisesta ostusoov:
Tasuda tuleb 16.8€.
```

Ülesanne 2. Hokiturniir (10 p)

Failis *hokiturniir.txt* on kirjas hokiturniiri tulemused võistkondade kaupa. Igal real on ühe hokivõistkonna mängude tulemused, kus "V" on võit ja "K" on kaotus (sellel turniiril viike ei ole). Tulemuste eraldajaks on semikoolon (;'). Võib eeldada, et igal võistkonnal on sama palju tulemusi. Ridade arv failis pole ette teada.

Näide faili *hokiturniir.txt* võimalikust sisust

```
K;K;K
V;V;K
V;V;V
K;K;V
```

Koostada funktsioon `loe_tulemused`, mis võtab argumendiks failinime ja tagastab kahemõõtmelise järjendi hokitulemuste kohta.

Näide funktsiooni `loe_tulemused` tööst

```
>>> loe_tulemused('hokiturniir.txt')
[['K', 'K', 'K'],
 ['V', 'V', 'K'],
 ['V', 'V', 'V'],
 ['K', 'K', 'V']]
```

Read on paigutatud ülevaatlikkuse mõttes üksteise alla.

Kirjuta funktsioon `kes_võitis`, mis võtab argumendiks kahemõõtmelise järjendi ja tagastab täisarvu, mitmes võistkond teenis kõige rohkem võite. Loendamine toimub selliselt, et kui kõige rohkem võite saavutas esimesel real olev võistkond, siis tagastab funktsioon täisarvu 1, kui teisel real olev võistkond, siis arvu 2 jne. Kui mitmel võistkonnal on võite sama palju, siis tagastatakse esimese sellise võistkonna järjekorranumber.

Näide funktsiooni `kes_võitis` tööst

```
>>> kes_võitis(['V', 'V', 'V'], ['K', 'K', 'K'], ['V', 'V', 'K'],
['K', 'K', 'V'])
1

>>> kes_võitis(['V', 'K', 'K'], ['K', 'V', 'V'], ['V', 'K', 'V'],
['V', 'K', 'K'])
2
```

Koosta programm, mis

- küsib kasutajalt failinime ja loeb andmed failist kahemõõtmelisse järjendisse, kasutades funktsiooni `loe_tulemused`,
- väljastab ekraanile, mitmes võistkond sai kõige rohkem võite, kasutades funktsiooni `kes_võitis`.

Näide programmi tööst (kasutaja sisend on paksus kaldkirjas)

```
Sisestage failinimi: hokiturniir.txt
Turniiri võitis 3. võistkond.
```

Programmeerimise 2. kontrolltöö

- Punktiline osa: kaks ülesannet.
 - Materjalide kasutamine on **lubatud**.
 - **Lahendused** ning **logid** laadida Moodle'isse, mõlemale on eraldi lingid.
 - Ilma logi esitamata lahendused arvesse ei lähe.
- Kokku on kontrolltööks aega **90 minutit**.
- Igasugune suhtlemine on **keelatud**.

Enne punktilise osaga alustamist

Kasuta programmeerimiseks Thonny! Palun ära kirjuta programmi teises IDEs ega kopeeri seda sealt Thonnysse! Sel juhul paistab logisid vaadates, et programm on kusagilt *copy-paste*'iga saadud, mis võib olla plagiaat. Veendu enne ülesandega alustamist, et logimine on sisse lülitatud (*Tools* → *Options* → *General* → *Log program usage events* või *Tööriistad* → *Seaded* → *Üldine* → *Logi kasutussündmusi*). Selleks, et logimine sujuks õigesti, **sulge enne ülesannetega alustamist Thonny ning käivita uuesti**. See tagab, et logisse ei jää üleliigset infot, kui Thonny on kauem avatud olnud.

Töö esitamine

Kui ülesanded on valmis, siis sulge korra Thonny ning käivita uuesti, et uusimad logid kindlasti salvestuksid. Esita programmid ja Thonny logifail(id). Programmidel võivad olla vabalt valitud nimed.

- Programmid esita Moodle'is lingi alt *2. kontrolltöö programmeerimise esitamine*.
- Logid leiad Thonny logifailide kaustast: ava menüü *Tools* → *Open Thonny data folder* (*Tööriistad* → *Ava Thonny andmekast*) ja seejärel ava kaust *user_logs*. Vali kõik failid, mis on tekkinud kontrolltöö ajal (kui vahepeal Thonny rohkem ei suletud, siis on neid faile vaid üks) ning esita need Moodle'is lingi alt *2. kontrolltöö logide esitamine*.

Ülesanne 1. Verekeskus (10 p)

Verekeskus vajab programmi, mis aitaks leida sobivaid doonoreid olemasolevast nimekirjast. Doonor on sobiv, kui ta on 18 kuni 60 aastat vana. Verekeskus hoiab potentsiaalsete doonorite nimekirja tekstifailis *doonorid.txt*. Iga inimese kohta on olemas komadega eraldatult järgmine info: nimi, telefon, vanus, veregrupp. Ridade arv failis pole ette teada.

Näide faili *doonorid.txt* võimalikust sisust

```
Juku Tamm,123123,31,A+
Triin Rebane,112233,33,B+
Anne Kott,332211,22,B+
Tom Henks,443311,22,A+
Maria Lill,223311,61,B-
```

Kirjuta funktsioon `rida_sõnastikuks`, mille argumendiks on faili rida sõnena ja mis tagastab sõnastiku, kus võtmeteks on sõned 'nimi', 'telefon', 'vanus', 'veregrupp' ja väärtusteks on andmed sisendsõnest.

Näide funktsiooni `rida_sõnastikuks` tööst

```
>>> rida_sõnastikuks("Juku Tamm,123123,31,A+")
{'nimi': 'Juku Tamm', 'telefon': '123123', 'vanus': 31,
 'veregrupp': 'A+'}
```

Kirjuta funktsioon `loe_failist`, mille argumendiks on faili nimi ja mis tagastab järjendi, mille elementideks on funktsiooniga `rida_sõnastikuks` moodustatud sõnastikud.

Näide funktsiooni `loe_failist` tööst

```
>>> loe_failist("doonorid.txt")
[{'nimi': 'Juku Tamm', 'telefon': '123123', 'vanus': 31,
 'veregrupp': 'A+'}, {'nimi': 'Triin Rebane', 'telefon': '112233',
 'vanus': 33, 'veregrupp': 'B+'}, {'nimi': 'Anne Kott', 'telefon':
 '332211', 'vanus': 22, 'veregrupp': 'B+'}, {'nimi': 'Tom Henks',
 'telefon': '443311', 'vanus': 22, 'veregrupp': 'A+'}, {'nimi':
 'Maria Lill', 'telefon': '223311', 'vanus': 61, 'veregrupp':
 'B-'}]
```

Kirjuta funktsioon `leia_doonorid`, mille argumentideks on funktsiooni `loe_failist` poolt tagastatud järjend ja veregrupp sõnena. Funktsioon kontrollib doonori vanust ja tagastab kaheelemendiliste ennikute järjendi. Iga enniku esimeseks elemendiks on doonori nimi ja teiseks tema telefon.

Näiteid funktsiooni `leia_doonorid` tööst

```
>>> andmed = loe_failist("doonorid.txt")
>>> leia_doonorid(andmed, 'A+')
[('Juku Tamm', '123123'), ('Tom Henks', '443311')]
>>> leia_doonorid(andmed, 'B-')
[]
```

Põhiprogrammis tuleb funktsiooni `loe_failist` abil sisse lugeda andmed failist *doonorid.txt*. Järgmiseks küsib programm kasutajalt, millise veregrupiga doonoreid on vaja. Programm väljastab sobivate doonorite nimed ja telefoninumbrid. Kui antud veregrupile sobivaid doonoreid ei ole, siis väljastatakse vastav teade. Veregruppe küsitakse seni, kuni kasutaja sisestab tühja sõne (""). Seejärel programm lõpetab töö.

Näide põhiprogrammi tööst (kasutaja sisend on paksus kirjas)

```
Millise veregrupiga doonoreid on vaja? A+
Leiti järgmised doonorid:
Nimi Juku Tamm, telefon 123123.
Nimi Tom Henks, telefon 443311.
Millise veregrupiga doonoreid on vaja? B-
Kahjuks veregrupiga B- doonorit ei leidu.
Millise veregrupiga doonoreid on vaja?
Programm lõpetas töö.
```

Ülesanne 2. Treeningpäevik (10 p)

Juku otsustas jälgida, mitu tundi päevas on ta füüsiliselt aktiivne. Treenitud tunnid märgib ta ujukomaarvudena faili *treening.txt*, kus igas reas on ühes nädalas treenitud tunnid tühikutega eraldatult. Esimene arv reas on esmaspäeval treenitud tunnid, teine arv on teisipäeval treenitud tunnid jne. Võib eeldada, et igas reas on täpselt 7 arvu. Ridade arv failis pole teada.

Näide faili *treening.txt* võimalikust sisust

```
1.5 2 3 1.5 2.1 0 2.5
1 2 2.5 2 3 1 2.5
3 3.5 1 0 1.5 2 0.5
0 0.5 0 1.5 0 3.5 0.5
```

Koosta funktsioon `loe_tunnid`, mis võtab argumendiks failinime ja tagastab kahemõõtmelise järjendi treeningtundide kohta. Kahemõõtmelise järjendi iga element on järjend ühel nädalal treenitud tundidega.

Näide funktsiooni `loe_tunnid` tööst

```
>>> loe_tunnid('treening.txt')
[[1.5, 2.0, 3.0, 1.5, 2.1, 0.0, 2.5], [1.0, 2.0, 2.5, 2.0, 3.0,
1.0, 2.5], [3.0, 3.5, 1.0, 0.0, 1.5, 2.0, 0.5], [0.0, 0.5, 0.0,
1.5, 0.0, 3.5, 0.5]]
```

Koosta funktsioon `kokku_nädalapäeval`, mis võtab argumendiks kahemõõtmelise järjendi ning nädalapäeva täisarvuna (1 - esmaspäev, 2 - teisipäev jne) ning tagastab selle nädalapäeva treeningtunnid kokku.

Näiteid funktsiooni `kokku_nädalapäeval` tööst

```
>>> kokku_nädalapäeval([[1.5, 2, 3, 1.5, 2.1, 0, 2.5], [1, 2, 2.5, 2, 3, 1, 2.5],
, [3, 3.5, 1, 0, 1.5, 2, 0.5], [0, 0.5, 0, 1.5, 0, 3.5, 0.5]], 1)
5.5
>>> kokku_nädalapäeval([[1.5, 2, 3, 1.5, 2.1, 0, 2.5], [1, 2, 2.5, 2, 3, 1, 2.5],
, [3, 3.5, 1, 0, 1.5, 2, 0.5], [0, 0.5, 0, 1.5, 0, 3.5, 0.5]], 2)
8.0
```

Koosta programm, mis

- küsib kasutajalt failinime ja loeb andmed failist kahemõõtmelisse järjendisse, kasutades funktsiooni `loe_tunnid`,
- väljastab nädalapäeva, millal treeniti kõige rohkem.

Näide põhiprogrammi tööst (kasutaja sisend on paksus kirjas)

```
Sisesta failinimi: treening.txt
Kõige rohkem treeniti 2. nädalapäeval - kokku 8.0 tundi.
```

Programmeerimise 2. kontrolltöö

- Punktiline osa: kaks ülesannet.
 - Materjalide kasutamine on **lubatud**.
 - **Lahendused** ning **logid** laadida Moodle'isse, mõlemale on eraldi lingid.
 - Ilma logi esitamata lahendused arvesse ei lähe.
- Kokku on kontrolltööks aega **90 minutit**.
- Igasugune suhtlemine on **keelatud**.

Enne punktilise osaga alustamist

Kasuta programmeerimiseks Thonny! Palun ära kirjuta programmi teises IDEs ega kopeeri seda sealt Thonnyse! Sel juhul paistab logisid vaadates, et programm on kusagilt *copy-paste*'iga saadud, mis võib olla plagiaat. Veendu enne ülesandega alustamist, et logimine on sisse lülitatud (*Tools* → *Options* → *General* → *Log program usage events* või *Tööriistad* → *Seaded* → *Üldine* → *Logi kasutussündmusi*). Selleks, et logimine sujus õigesti, **sulge enne ülesannetega alustamist Thonny ning käivita uuesti**. See tagab, et logisse ei jää üleliigset infot, kui Thonny on kauem avatud olnud.

Töö esitamine

Kui ülesanded on valmis, siis sulge korra Thonny ning käivita uuesti, et uusimad logid kindlasti salvestuksid. Esita programmid ja Thonny logifail(id). Programmidel võivad olla vabalt valitud nimed.

- Programmid esita Moodle'is lingi alt *2. kontrolltöö programmeerimise esitamine*.
- Logid leiad Thonny logifailide kaustast: ava menüü *Tools* → *Open Thonny data folder* (*Tööriistad* → *Ava Thonny andmekast*) ja seejärel ava kaust *user_logs*. Vali kõik failid, mis on tekkinud kontrolltöö ajal (kui vahepeal Thonnyt rohkem ei suletud, siis on neid faile vaid üks) ning esita need Moodle'is lingi alt *2. kontrolltöö logide esitamine*.

Ülesanne 1. Orienteerumisvõistlus (10 p)

Orienteerumisklubi Kuulsused korraldab orienteerumisvõistluse, mille jaoks salvestati eelregistreerunud võistlejate andmed faili *võistlejad.txt*. Faili ühel real on komadega eraldatult võistleja nimi, võistlusklass (M, N või J) ja SI-kaardi number (võistlemiseks vajaliku kiibi kood). Viimase asemel võib olla ka RENT. Ridade arv failis pole ette teada.

Näide faili *võistlejad.txt* võimalikust sisust

```
Andrus Ansip,M,89898989
Kersti Kaljulaid,N,79797979
Mari Maasikas,J,RENT
Märt Möttus,J,84000034
Ivo Linna,M,RENT
Jürgen Ligi,M,84638243
```

Kirjuta funktsioon `loe_failist`, mille argumendiks on faili nimi ja mis tagastab sõnastiku, kus võtmeks on võistleja nimi ning väärtuseks on kaheelemendiline ennik. Enniku esimene element on võistlusklass sõnena ja teine element on SI-kaardi number sõnena (või sõne "RENT").

Näide funktsiooni `loe_failist` tööst

```
>>> loe_failist("võistlejad.txt")
{'Andrus Ansip': ('M', '89898989'), 'Kersti Kaljulaid': ('N',
'79797979'), 'Mari Maasikas': ('J', 'RENT'), 'Märt Möttus': ('J',
'84000034'), 'Ivo Linna': ('M', 'RENT'), 'Jürgen Ligi': ('M',
'84638243')}
```

Koosta funktsioon `võistlusklassis`, mille argumentideks on funktsiooni `loe_failist` abil loodud sõnastik ja võistlusklass sõnena. Funktsioon peab tagastama järjendi antud võistlusklassis osalevate sportlaste nimedega.

Näide funktsiooni `võistlusklassis` tööst

```
>>> d = loe_failist("võistlejad.txt")
>>> võistlusklassis(d, "M")
['Andrus Ansip', 'Ivo Linna', 'Jürgen Ligi']
```

Koosta funktsioon `osalustasu`, mille argumentideks on funktsiooni `loe_failist` abil loodud sõnastik ja võistleja nimi. Funktsioon peab tagastama võistleja osalustasu, mis arvutatakse järgmiselt. Võistlusklassis J maksab võistlemine 3€ ning teistes võistlusklassides 5€. Kui võistleja SI-kaardi number on "RENT", siis lisandub osalusele veel täiendavalt 1€. Kui võistleja nime sõnastikus ei ole, siis tagastatakse 0.

Näiteid funktsiooni `osalustasu` tööst

```
>>> d = loe_failist("võistlejad.txt")
>>> osalustasu(d, "Mari Maasikas")
4
>>> osalustasu(d, "Ivo Linna")
6
>>> osalustasu(d, "Ivo Lina")
0
```

Põhiprogrammis tuleb funktsiooni `loe_failist` abil sisse lugeda andmed failist *võistlejad.txt*. Kõigepealt küsitakse kasutajalt, millise võistlusklassi võistleja osalustasu ta soovib maksta. Seejärel kuvatakse kasutajale tema poolt sisestatud võistlusklassi võistlejate nimekiri. Kui võistlusklassis ei ole võistlejaid, siis antakse vastav teade ja küsitakse võistlusklassi uuesti. Seejärel küsitakse kasutajalt, kelle osalustasu ta soovib maksta. Kui võistleja on võistlusklassis olemas, arvutab programm võistleja osalustasu ja väljastab selle. Kui sisestatud võistleja nime võistlusklassis ei ole, siis teavitab programm kasutajat ja küsib uuesti võistlusklassi nime. Programm jätkab tööd, kuni võistlusklassi nime asemel sisestatakse tühi sõne (""). Seejärel väljastab programm kõikide sisestatud võistlejate osalustasude summa.

Näide põhiprogrammi tööst (kasutaja sisend on paksus kirjas)

```
Sisesta võistlusklass M/N/J: M
Võistlusklassis M võistlevad järgmised sportlased:
Andrus Ansip
Ivo Linna
Jürgen Ligi
Kelle osaluse eest soovid tasuda? Ivo Lina
Sellist võistlejat ei ole võistlusklassis M registreeritud.
Sisesta võistlusklass M/N/J: M
Võistlusklassis M võistlevad järgmised sportlased:
Andrus Ansip
Ivo Linna
Jürgen Ligi
Kelle osaluse eest soovid tasuda? Ivo Linna
Võistleja Ivo Linna osalus maksab 6€.
Sisesta võistlusklass M/N/J: S
Selles võistlusklassis ei ole võistlejaid registreeritud.
Sisesta võistlusklass M/N/J: J
Võistlusklassis J võistlevad järgmised sportlased:
Mari Maasikas
Märt Möttus
Kelle osaluse eest soovid tasuda? Märt Möttus
Võistleja Märt Möttus osalus maksab 3€.
Sisesta võistlusklass M/N/J:
Kokku tuleb tasuda 9€.
```

Ülesanne 2. Lihtsustatud Connect 4 seisu hindamine (10 p)

Connect 4 on mäng, kus kaks mängijat kukutavad kordamööda oma nuppe (žetoone) vertikaalsesse mänguplatsi mängija poolt oma käigul valitud veergu. Mänguplatsil olevad nupud moodustavad ridu, veerge ja diagonaale. Mängija eesmärgiks on saada 4 oma nuppu kõrvuti kas horisontaalselt, vertikaalselt või diagonaalselt. Jukule meeldib oma sõpradega mängida lihtsustatud Connect 4 mängu, kus võitmiseks on vaja saada 4 oma nuppu üksteise peale samasse veergu. Juku otsustas mängu seisu hindamiseks kirjutada programmi. Võib eeldada, et mänguplats on alati mõõtmetega 4x4. Mängijate nuppe mänguplatsi lahtris tähistatakse "X" ja "O". Nupu puudumist tähistab "-". Mänguplats esitatakse maatriksina. Koosta funktsioon `väljasta_laud`, mis võtab argumentiks mänguplatsi maatriksi ja väljastab selle ilusal kujul. See tähendab, et maatriksi iga rida on eraldi real ning elemendid on joondatult üksteise all.

Näide funktsiooni `väljasta_laud` tööst

```
>>> väljasta_laud([["-", "-", "X", "O"], ["X", "-", "O", "O"],
["X", "O", "X", "X"], ["X", "X", "O", "X"]])
- - X O
X - O O
X O X X
X X O X
```

Koosta funktsioon `kontrolli_laud`, mis võtab argumendiks maatriksi, mis kujutab mängu seis. Funktsioon tagastab "X", kui võitjaks osutub mängija nuppudega "X", ja "O" kui võitjaks on mängija nuppudega "O". Kui võitjat veel pole, siis tuleb tagastada sõne "Võitjat pole.". Mängija osutub võitjaks siis, kui tal on 4 kõrvutiseisvat nuppu ühes veerus.

Näited funktsiooni `kontrolli_laud` tööst

```
kontrolli_laud([["-", "-", "X", "O"], ["X", "-", "O", "O"], ["X", "O", "X", "X"], ["X", "X", "X", "X"]])
'Võitjat pole.'

kontrolli_laud([["-", "-", "O", "-"], ["-", "-", "O", "-"], ["-", "-", "O", "-"], ["-", "O", "-", "-"], ["-", "-", "O", "-"]])
'O'
```

Koosta funktsioon `kas_mäng_läbi`, mis võtab argumendiks maatriksi, mis iseloomustab mängu seis. Funktsioon tagastab tõeväärtuse `True` või `False` vastavalt sellele, kas mäng on läbi või mitte. Mäng on läbi, kui kõik mänguplatsi lahtrid on nuppe täis (mänguplatsil ei ole ühtegi "-").

Näited funktsiooni `kas_mäng_läbi` tööst

```
kas_mäng_läbi([["O", "O", "X", "O"], ["X", "O", "O", "O"], ["X", "O", "X", "X"], ["X", "X", "O", "X"]])
True

kas_mäng_läbi([["-", "-", "O", "-"], ["-", "-", "O", "-"], ["-", "-", "O", "-"], ["-", "O", "-", "-"], ["-", "-", "O", "-"]])
False
```

Põhiprogrammis loo muutujasse mänguvälja maatriks ja väljasta see. Kasutades funktsioone `kontrolli_laud` ja `kas_mäng_läbi`, väljasta, kas keegi võitis või on mäng veel pooleli.

Näide põhiprogrammi tööst

Näites on etteantud maatriksi väärtus `m = [["-", "O", "-", "-"], ["-", "X", "X", "-"], ["X", "O", "X", "X"], ["X", "X", "O", "X"], ["X", "X", "O", "X"], ["X", "X", "O", "X"], ["X", "X", "O", "X"]]`.

```
- O - -
- X X -
X O X X
X X O X
Võitjat pole.
Mäng on veel pooleli!
```